

Modern Livestock Management System

Detailed System Overview, Software Logic, Backend Workflow, Database Structure, and Implementation Guide

Prepared for concept design, software architecture, prototyping, and Claude-assisted development

Date: 2026-05-15

Version 1.0

Table of Contents

1. 1. Executive overview
2. 2. System purpose and operating principle
3. 3. End-to-end architecture
4. 4. How data moves through the platform
5. 5. User roles and access model
6. 6. Core modules and how each one works
7. 7. Backend logic and service responsibilities
8. 8. Database design and entity structure
9. 9. IoT, MQTT, LoRaWAN, edge gateway, and offline behavior
10. 10. AI and decision-support logic
11. 11. Alert engine and task workflow
12. 12. Mobile app, web dashboard, and user experience
13. 13. Security, privacy, auditability, and data protection
14. 14. Deployment, DevOps, observability, and maintenance
15. 15. Implementation roadmap
16. 16. Claude-ready system prompt
17. 17. Appendices: schemas, API examples, event payloads, and references

1. Executive Overview

This document explains how a modern livestock management platform should work from end to end. The system is designed to move livestock farming from manual observation and paper records into a connected, data-driven operating model. It combines animal identification, health records, breeding management, feeding and water monitoring, geofencing, weighing, production tracking, financial management, AI predictions, and alerts into one platform.

The system should not only track animal location. It should identify each animal, record its full life history, monitor the farm environment, capture sensor data, detect abnormal behavior, recommend actions, and generate reports that improve productivity, welfare, traceability, and profitability.

Goal	Meaning in the system
Prevent losses	Detect sickness, theft, water failure, feed shortages, and breeding problems early.
Improve productivity	Track weight gain, reproduction, milk yield, feed efficiency, and herd performance.
Improve decision-making	Convert raw sensor and farm records into alerts, tasks, analytics, and recommendations.
Support rural deployment	Use offline mobile workflows, LoRaWAN, GSM, solar-powered devices, and low-bandwidth messaging.
Support growth	Begin with RFID and records, then scale into IoT sensors, AI, cameras, drones, and traceability.

Source notes: Precision livestock management references [1], [2].

2. System Purpose and Operating Principle

The platform should operate as the digital nervous system of the farm. Every animal, device, location, worker action, treatment, breeding event, sale, and sensor reading becomes a structured digital event. The backend stores these events, applies rules and AI models, then produces actions.

2.1 What the system should answer

- Which animals are sick, at risk, missing, pregnant, underperforming, or ready for sale?
- Which animals require vaccination, deworming, follow-up treatment, or pregnancy checking?
- Which paddocks are being overgrazed and which ones require recovery?
- Which water points, feed stocks, gates, scales, sensors, or gateways need attention?
- Which animals or groups generate profit, and which ones consume too much feed or medicine?
- What action should a worker, farmer, manager, or vet take today?

2.2 Core operating loop

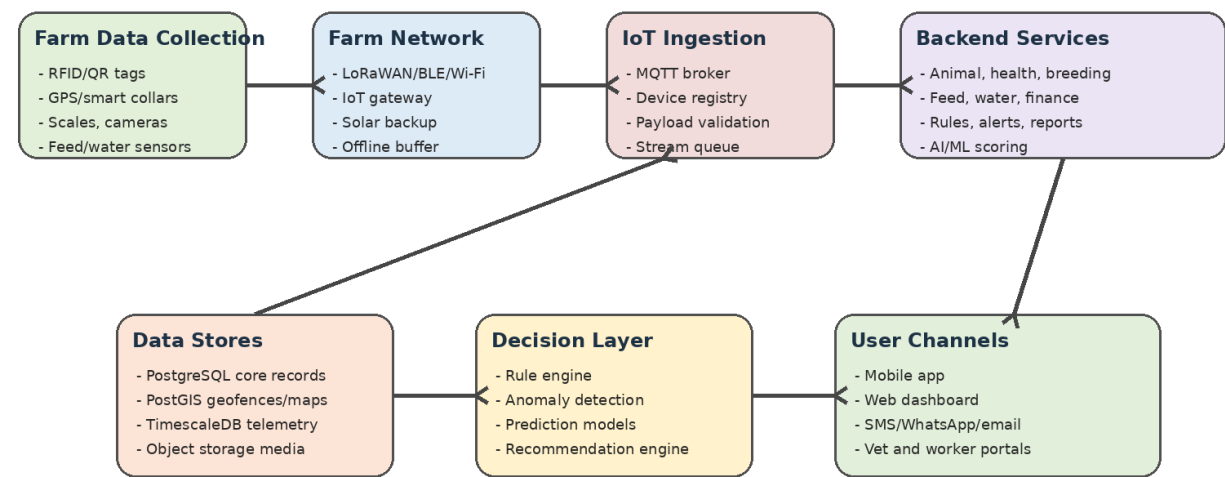
18. Capture data from animals, equipment, workers, vets, sensors, cameras, and external systems.
19. Validate the data by checking device identity, user permission, timestamps, units, and value ranges.
20. Store the data in the correct data store: relational records, time-series telemetry, geospatial tables, or object storage.
21. Analyze the new event using thresholds, rules, historical baselines, geofence logic, and AI models.

- 22. Create an alert, task, dashboard update, report entry, or recommendation.
- 23. Record the human response so the system becomes more accurate over time.

3. End-to-End Architecture

The architecture should be modular so the system can start as a farm-record platform and later grow into a full IoT and AI system.

End-to-End Livestock Management System Architecture



Main principle: collect data at the farm, validate it at ingestion, store it correctly, analyze it, and turn it into alerts, tasks, reports

Source notes: LoRaWAN references [3], [4], MQTT 5.0 reference [5], PostGIS [6], TimescaleDB [7], Kafka [8].

3.1 Architecture layers

Layer	Purpose	Recommended technologies
Farm device layer	Collects animal, equipment, environment, feed, water, and movement data.	RFID, QR tags, GPS collars, smart ear tags, accelerometers, scales, cameras, water and feed sensors.
Farm network layer	Moves data from devices to the platform under rural connectivity constraints.	LoRaWAN, BLE, Wi-Fi, GSM/4G, NB-IoT where available, satellite for remote sites.
Gateway/edge layer	Buffers data, converts protocols, performs local validation, and keeps critical alerts working offline.	Industrial gateway, Raspberry Pi/Jetson for prototype, MQTT client, local SQLite cache.
Ingestion layer	Receives telemetry and events safely and at scale.	MQTT broker, API gateway, stream processing, schema validation.
Backend services	Runs business logic for animals, health, breeding, finance, alerts, reporting, and users.	Microservices or modular monolith using Node.js/NestJS, Python/FastAPI, Java/Spring, or Go.
Data layer	Stores structured records, geospatial data, telemetry, media, cache, and audit logs.	PostgreSQL, PostGIS, TimescaleDB, object storage, Redis.
AI and analytics layer	Predicts health risks, heat, weight gain, theft risk, and feed/water needs.	Python ML services, feature store, model registry, edge inference for cameras.
Application layer	Gives farmers, workers, vets, admins, and	Flutter/React Native mobile app,

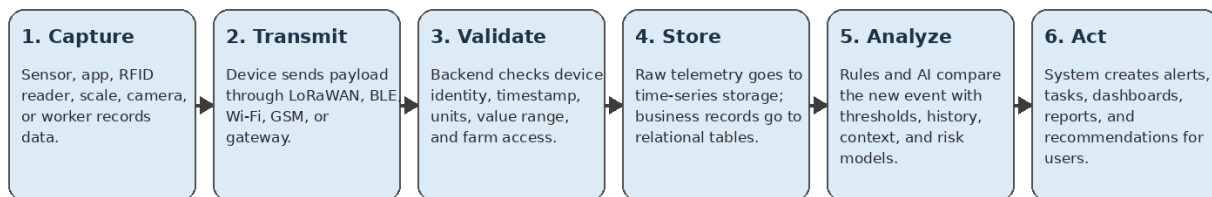
Layer	Purpose	Recommended technologies
	buyers usable interfaces.	React/Next.js web dashboard, SMS/WhatsApp/email.

3.2 Why this architecture is suitable

- It supports low-connectivity farms by allowing gateways and mobile apps to cache data locally.
- It separates high-volume telemetry from normal business records, avoiding slow database queries as sensors increase.
- It makes geofencing and map logic first-class by using geospatial storage, not only plain latitude and longitude columns.
- It allows AI to be added gradually without rebuilding the entire system.
- It gives different users different portals while keeping one consistent backend and database model.

4. How Data Moves Through the Platform

Sensor-to-Action Data Flow



Feedback loop:

When a farmer, worker, or vet confirms or rejects an alert, that feedback is stored and used to tune thresholds, retrain models, and improve future recommendations.

4.1 Example: water shortage workflow

24. A water-level sensor at Paddock B reads 8 percent remaining.
25. The sensor sends the reading through LoRaWAN to the farm gateway.
26. The gateway publishes an MQTT message to the cloud when connectivity exists, or stores it locally if offline.
27. The ingestion service validates the device identity, timestamp, farm ID, and reading range.
28. The telemetry store records the raw reading.
29. The rules engine checks thresholds and sees that the value is below the critical limit.
30. The alert service creates a critical alert and a worker task to inspect or refill the water point.
31. The notification service sends a push notification and SMS/WhatsApp message.
32. A worker completes the task and uploads a photo as evidence.

33. The system closes the alert, records response time, and updates the farm report.

4.2 Example: possible illness workflow

34. Animal COW-017 has reduced movement, reduced drinking, and a high body-temperature reading.

35. The backend links these readings to the animal profile and current herd group.

36. The AI risk model compares today's data with the animal's normal behavior and herd averages.

37. The system creates a high-priority health-risk alert with likely causes and recommended checks.

38. A worker is assigned to inspect the animal and records visible symptoms using the mobile app.

39. The vet reviews the animal's history and records treatment.

40. The system calculates drug withdrawal periods and follow-up dates.

41. The animal status changes to Under Treatment, then returns to Normal once recovered.

5. User Roles and Access Model

The system must use role-based access control. Each user should only see the features needed for their work.

Role	Main permissions	Restricted areas
Farm owner	Full access to farm records, finance, reports, users, alerts, sales, and configuration.	Cannot access other farms unless explicitly linked as an organization owner.
Farm manager	Manage animals, workers, health, breeding, feeding, water, tasks, reports, and devices.	May have limited financial or user-management permissions depending on owner settings.
Worker	View assigned tasks, scan animals, add field notes, record feeding, water, weight, and observations.	No access to sensitive finance, system configuration, or unrelated farms.
Veterinarian	View health history, record diagnosis, treatment, vaccination, prescriptions, and follow-ups.	No access to financial data unless explicitly permitted.
Admin/support	Support device setup, technical configuration, and issue resolution.	Should use audited support access with farm owner consent.
Buyer/abattoir/inspector	View selected traceability records for animals being sold or inspected.	No broad access to the whole farm database.

5.1 Access rules that should be enforced

- Every query must be scoped by farm_id or organization_id to prevent data leakage between farms.
- Sensitive actions such as deleting animals, changing ownership, exporting financial data, or changing device assignment should require elevated permission.
- All record changes should be written to an audit log with user ID, timestamp, old value, new value, IP/device information, and reason where applicable.
- Offline mobile changes should retain the identity of the user and device that created the change, even if synchronized later.

6. Core Modules and How Each One Works

6.1 Farm and organization setup module

This module creates the digital structure of the farm. It stores farm boundaries, paddocks, kraals, gates, water points, feeding points, staff, animal types, device inventory, and operating rules.

- Create organization, farm, and branch records.
- Draw or upload farm boundaries and paddock polygons.
- Register water points, feed points, gates, sheds, barns, dip tanks, and crush pens.
- Configure default health schedules, vaccination calendars, alert thresholds, and user roles.
- Support multiple farms under one owner or company.

6.2 Animal identity and profile module

This module creates a digital passport for every animal. Every event in the animal's life should connect back to this profile.

- Register RFID, QR tag, visual tag, GPS device ID, species, breed, sex, color, age, parentage, status, photos, and purchase/source records.
- Support animal groups such as herds, flocks, pens, paddocks, age groups, production groups, or treatment groups.
- Show a full timeline: birth, purchase, vaccination, treatment, weight, breeding, production, movement, sale, death, or transfer.
- Allow bulk import by CSV or spreadsheet for farms that already have records.

6.3 Device and IoT module

This module manages sensors, gateways, collars, readers, scales, and cameras. Its job is to keep farm hardware trustworthy and visible.

- Register every device with serial number, device type, farm, assigned animal or location, firmware version, battery level, signal strength, and last seen time.
- Detect devices that stop reporting, send impossible values, have low battery, or move outside expected locations.
- Support firmware/version tracking and safe device replacement without losing animal history.
- Use secure credentials per device rather than one shared farm credential.

6.4 Health monitoring module

This module manages illness, treatment, observation, quarantine, recovery, mortality, and welfare.

- Record symptoms, diagnosis, medicine, dose, vet, treatment date, follow-up date, withdrawal period, photos, and outcome.
- Combine manual observations with sensor signals such as temperature, movement, rumination, water intake, feeding, and weight change.
- Automatically flag health-risk patterns and escalate unresolved health alerts.
- Allow vets to review history before treating an animal.

6.5 Vaccination and preventive care module

This module turns preventive care into schedules, reminders, and compliance records.

- Create species-specific vaccination and deworming schedules.
- Notify users before due dates and escalate overdue events.
- Store vaccine batch number, disease, date given, next due date, administered by, and proof image if needed.
- Calculate medicine withdrawal periods for milk, meat, or sale restrictions.

6.6 Breeding and reproduction module

This module manages heat detection, mating, artificial insemination, pregnancy, births, parentage, fertility, and reproductive performance.

- Detect heat manually or from increased activity/restlessness/mounting behavior where sensors exist.
- Record service date, sire, method, technician, semen batch, pregnancy check, expected birth date, and outcome.
- Automatically create reminders for pregnancy check, calving/kidding/lambing/farrowing, and post-birth care.
- Track fertility metrics such as conception rate, calving interval, open days, and repeat breeders.

6.7 Feeding and nutrition module

This module controls feed inventory, feeding plans, feed usage, feed cost, and feed efficiency.

- Record feed purchases, type, quantity, cost, expiry date, supplier, and storage location.
- Use bin sensors or load cells to estimate stock remaining.
- Create ration plans by animal group, weight, production target, and feed availability.
- Calculate feed conversion ratio, cost per animal, cost per kg gained, or feed cost per litre of milk.

6.8 Water monitoring module

This module ensures animals have reliable and safe water.

- Monitor tank/trough level, pump status, water flow, leak patterns, and quality values such as pH, turbidity, temperature, or salinity.
- Alert when water is low, flow is abnormal, pump status changes, or drinking behavior drops unexpectedly.
- Link water points to paddocks and animal groups so the system knows which animals are affected.
- Keep water history for maintenance planning and disease investigation.

6.9 Weight and growth module

This module tracks animal performance and market readiness.

- Accept manual weights, digital scale readings, or walk-over weighing events.
- Calculate average daily gain, growth curve, expected market date, and underperformance alerts.
- Compare animals by breed, age, feed group, paddock, and health history.
- Recommend inspection when weight gain drops sharply or does not match feed intake.

6.10 Production module

This module is used mainly for dairy, wool, eggs, meat batches, or other production outputs.

- Record milk yield, lactation stage, milk quality, mastitis indicators, wool clips, egg output, or batch production.
- Calculate production per animal, per group, per day, and per feed cost.
- Flag sudden drops that may indicate illness, stress, feed issues, or water problems.
- Support production reports for owners, buyers, processors, or inspectors.

6.11 Grazing and pasture module

This module manages land, paddocks, carrying capacity, rotational grazing, and pasture recovery.

- Store paddock polygons, stocking rate, grazing start/end dates, recovery period, water points, and pasture notes.
- Use satellite, drone, camera, or manual scoring to estimate pasture condition where available.
- Alert when overgrazing risk is high or animals should be moved.
- Plan rotational grazing schedules and compare grazing pressure across paddocks.

6.12 Security and theft prevention module

This module protects animals and farm infrastructure.

- Create geofences for farm boundary, paddocks, quarantine zones, sale pens, and restricted areas.
- Detect animals leaving allowed areas, unusual night movement, gate openings, device tampering, or missing animals.
- Combine GPS, RFID gate readers, gate sensors, and cameras for stronger theft detection.
- Generate exportable incident reports with last known location, timestamp, map screenshot, device data, and owner details.

6.13 Finance and profitability module

This module connects animal records to money.

- Record purchases, sales, feed costs, medicine costs, vet costs, labor, transport, breeding costs, and equipment expenses.
- Calculate profit per animal, group, species, paddock, production system, or season.
- Show animals that cost more than they produce.
- Export reports for farm owners, investors, lenders, cooperatives, or insurers.

6.14 Reports and analytics module

This module converts operational data into management information.

- Generate health, vaccination, mortality, breeding, growth, production, feed, water, financial, security, and worker-performance reports.
- Export PDF, Excel, and CSV files.
- Allow report filters by species, breed, age group, paddock, date range, animal group, worker, or event type.

- Schedule weekly or monthly owner reports.

7. Backend Logic and Service Responsibilities

The backend can be built either as a modular monolith for the first version or as microservices for a larger commercial deployment. The important principle is that each domain must have clear ownership and clean interfaces.

Service/domain	Responsibilities
Identity and access service	Users, roles, sessions, permissions, organization/farm access, audit identity.
Farm service	Farms, paddocks, locations, water points, feed points, gates, farm settings.
Animal service	Animal profiles, groups, lifecycle status, ownership, photos, tag assignments.
Device service	Device registry, device credentials, assignment, status, battery, firmware, last seen.
Telemetry ingestion service	Receives sensor payloads, validates schema, normalizes units, stores raw telemetry, publishes events.
Health service	Symptoms, diagnoses, treatments, medicine, withdrawal periods, quarantine, mortality.
Breeding service	Heat, mating, AI, pregnancy, births, offspring linking, fertility metrics.
Feed and water service	Feed inventory, feeding plans, water points, consumption, quality and shortage logic.
Weight and production service	Weight records, growth metrics, production outputs, performance analytics.
Geofence service	Location processing, geofence checks, paddock membership, route and movement history.
Rules engine	Thresholds, schedules, due dates, escalation rules, critical condition detection.
AI service	Feature extraction, risk scores, prediction models, anomaly detection, recommendations.
Alert and task service	Alert creation, assignment, status changes, escalation, worker tasks, SLA tracking.
Notification service	Push, SMS, WhatsApp, email, in-app notifications, retries, delivery logs.
Reporting service	PDF/Excel/CSV reports, dashboards, scheduled reports, export jobs.
Sync service	Offline mobile synchronization, conflict detection, idempotency, retries.
Audit service	Immutable logs of sensitive operations, access events, device events, and record changes.

7.1 Event-driven backend behavior

Every major action should create an event. This makes the system reactive, auditable, and easier to extend.

Event	Producer	Consumers
animal.registered	Animal service	Audit service, reporting service, traceability module.

Event	Producer	Consumers
device.telemetry.received	Telemetry ingestion	Rules engine, AI service, dashboard, time-series store.
water.low_detected	Rules engine	Alert service, task service, notification service.
animal.geofence_breach	Geofence service	Security module, alert service, notification service.
health.risk_detected	AI service	Alert service, vet portal, task service.
vaccination.due	Rules engine	Alert service, calendar/task service.
animal.treated	Health service	Withdrawal-period logic, audit, reporting.
animal.sold	Finance service	Traceability report, animal status update, finance dashboard.

7.2 Backend logic for incoming telemetry

42. Receive payload from MQTT, HTTP API, gateway batch upload, or mobile sync.
43. Authenticate the device or gateway and verify it belongs to the farm.
44. Validate payload schema, timestamp, unit, sensor type, and expected value range.
45. Normalize units such as litres, kilograms, degrees Celsius, percentage, or coordinates.
46. Store raw payload for audit/debugging and normalized reading for analytics.
47. Attach reading to animal_id, device_id, farm_id, paddock_id, or water_point_id.
48. Publish a domain event for rules and AI processing.
49. Run threshold checks, geofence checks, anomaly checks, and schedule checks.
50. Create alerts/tasks/notifications if required.
51. Update dashboards and aggregates such as latest water level, latest location, and battery status.

8. Database Design and Entity Structure

The data model should separate core business records, high-volume telemetry, geospatial features, files/media, and audit logs. This prevents sensor data from slowing down animal profiles and reports.

Storage type	Use	Example data
PostgreSQL relational database	Core transactional records.	Users, farms, animals, health records, breeding, finance, tasks, alerts.
PostGIS extension	Geospatial logic and map queries.	Farm boundaries, paddock polygons, animal locations, geofence checks.
TimescaleDB or time-series store	High-volume telemetry and historical sensor readings.	Water level every 5 minutes, animal movement, temperature, weight readings, environment readings.
Object storage	Unstructured files and media.	Animal photos, vet documents, certificates, camera images, PDF reports.
Redis/cache	Fast temporary data and real-time state.	Latest device state, alert counters, sessions, dashboard cache.
Search/index store optional	Fast text search and filtering at scale.	Animal names/tags, documents, notes, diagnostic text.

Source notes: PostGIS [6], TimescaleDB [7].

8.1 Core database tables

Table	Key fields	Purpose
organizations	organization_id, name, owner_user_id, status, created_at	Parent entity for companies/cooperatives managing multiple farms.
farms	farm_id, organization_id, name, location, timezone, boundary_polygon, status	Represents a farm or ranch.
users	user_id, full_name, phone, email, password_hash, status	Stores user identity.
farm_memberships	farm_id, user_id, role, permissions, status	Controls access to farm data.
animals	animal_id, farm_id, tag_number, rfid_code, species, breed, sex, dob, status	Main animal profile table.
animal_groups	group_id, farm_id, name, species, purpose, location_id	Herds, flocks, pens, paddock groups, production groups.
animal_group_memberships	animal_id, group_id, start_date, end_date	Tracks group membership history.
devices	device_id, farm_id, serial_number, type, status, last_seen_at, battery_level	All farm devices and sensors.
device_assignments	assignment_id, device_id, animal_id/location_id, start_at, end_at	Links devices to animals or locations over time.
locations	location_id, farm_id, type, name, geometry, metadata	Paddocks, water points, gates, barns, kraals, feed points.
events	event_id, farm_id, animal_id, event_type, payload_json, occurred_at	Domain event log for auditing and integrations.
tasks	task_id, farm_id, assigned_to, title, priority, due_at, status	Work items created manually or automatically.
alerts	alert_id, farm_id, animal_id, type, severity, message, status, created_at	System-generated warnings and reminders.

8.2 Animal lifecycle and production tables

Table	Key fields	Purpose
health_records	health_record_id, animal_id, symptoms, diagnosis, medicine, dose, vet_id, outcome	Treatment and disease history.
vaccination_records	vaccination_id, animal_id, vaccine_name, batch_no, date_given, next_due_date	Preventive health records.
medicine_withdrawals	withdrawal_id, animal_id, product_type, start_date, end_date	Milk/meat sale restriction periods.
breeding_records	breeding_id, female_animal_id, male_animal_id, method, service_date, status	Mating and AI records.
pregnancy_records	pregnancy_id, animal_id, check_date, result, expected_birth_date	Pregnancy tracking.
birth_records	birth_id, mother_id, offspring_id, birth_date, outcome, notes	Birth and parent-child linking.
weight_records	weight_id, animal_id, weight_kg, method, recorded_at, recorded_by	Weight history.
production_records	production_id, animal_id/group_id, production_type, quantity, unit, recorded_at	Milk, wool, egg, or batch production.
movement_records	movement_id, animal_id/group_id, from_location_id, to_location_id, moved_at	Paddock/farm movement history.
sales_records	sale_id, animal_id/group_id, buyer, sale_price, sale_date, documents	Animal and production sales.
mortality_records	mortality_id, animal_id, death_date, cause,	Death and loss records.

Table	Key fields	Purpose
	disposal_method, verified_by	

8.3 Feed, water, finance, and reporting tables

Table	Key fields	Purpose
feed_inventory	feed_id, farm_id, type, quantity, unit, cost_per_unit, expiry_date	Feed stock and cost tracking.
feeding_plans	plan_id, farm_id, group_id, ration_json, start_date, end_date	Planned rations by group.
feeding_records	record_id, farm_id, group_id/animal_id, feed_id, quantity, recorded_at	Actual feeding events.
water_points	water_point_id, farm_id, location_id, capacity_litres, source_type	Fixed water assets.
water_quality_records	record_id, water_point_id, pH, turbidity, temperature, salinity, recorded_at	Water quality history.
financial_transactions	transaction_id, farm_id, animal_id/group_id, category, amount, type, date	Income and expenses.
report_jobs	job_id, farm_id, report_type, parameters_json, status, output_url	Scheduled/generated reports.
audit_logs	audit_id, farm_id, user_id, action, entity, before_json, after_json, timestamp	Security and compliance audit trail.

8.4 Time-series telemetry schema

High-volume sensor readings should be stored as time-series hypertables or partitioned tables. This is important because telemetry grows faster than normal farm records.

Column	Type	Notes
time	timestamp with timezone	Primary time dimension. Use device time plus server received time.
farm_id	uuid	Required for data isolation.
device_id	uuid	Device producing the reading.
animal_id	uuid nullable	Set when reading belongs to an animal.
location_id	uuid nullable	Set when reading belongs to a fixed farm point.
metric	text	Example: body_temperature, water_level, battery, gps, weight, humidity.
value_num	double precision	Numeric value where applicable.
unit	text	kg, C, %, litres, m/s, ppm, etc.
latitude/longitude or geometry	geography/geometry nullable	Used for GPS and spatial readings.
quality_flag	text	valid, suspect, missing, duplicate, estimated.
raw_payload	jsonb	Original payload for debugging and traceability.

9. IoT, MQTT, LoRaWAN, Edge Gateway, and Offline Behavior

The IoT design must support low-power sensors, long-range communication, intermittent internet, and low-cost rural deployment.

9.1 Recommended communication pattern

- Use LoRaWAN for long-range low-power farm sensors such as water level, gate status, temperature, and pasture points.
- Use BLE for short-range wearable-to-phone or wearable-to-gateway communication.
- Use Wi-Fi inside barns, dairy sheds, offices, or areas with existing farm internet.
- Use GSM/4G from the gateway to the cloud when mobile coverage is available.
- Use MQTT 5.0 for lightweight publish/subscribe IoT messaging from gateways and devices to the backend.
- Use HTTP REST or GraphQL for mobile and web application APIs.

Source notes: LoRaWAN [3], [4], MQTT 5.0 [5].

9.2 Gateway responsibilities

- Receive data from LoRaWAN, BLE, serial, Modbus, Wi-Fi, or USB-connected devices.
- Buffer data locally during internet outages.
- Perform basic validation to reject obviously corrupted readings.
- Normalize timestamps and attach farm/gateway identifiers.
- Publish data to MQTT topics when online.
- Run local critical rules where possible, such as water empty, gate opened, or freezer/cold-chain alarm.
- Expose local diagnostics for battery, signal strength, sensor health, and gateway uptime.

9.3 Suggested MQTT topic structure

```
farm/{farmId}/device/{deviceId}/telemetry
farm/{farmId}/device/{deviceId}/status
farm/{farmId}/gateway/{gatewayId}/heartbeat
farm/{farmId}/animal/{animalId}/location
farm/{farmId}/alert/{alertId}/ack
farm/{farmId}/command/{deviceId}/downlink
```

9.4 Offline-first mobile behavior

The mobile app should continue working even with no internet. This is critical for rural farms.

Offline action	How it should work
Animal registration	Create a temporary local ID, scan RFID/QR, capture photos, then sync and replace with server ID later.
Health/treatment entry	Store locally with user, timestamp, animal ID, medicine, dosage, and images. Sync when online.
Task completion	Allow workers to mark tasks complete offline; preserve proof photos and GPS/time metadata.
Conflict handling	Use version numbers and change logs. Merge non-conflicting edits and flag conflicting edits for manager review.
Critical alerts	Show cached and locally generated alerts where possible, then

Offline action	How it should work
	reconcile with backend when online.

10. AI and Decision-Support Logic

AI should support the farmer, not replace the farmer or veterinarian. The system should give risk scores, explanations, and recommended next actions.

10.1 AI model categories

Model/use case	Inputs	Output
Health risk scoring	Temperature, movement, water intake, feeding, weight change, previous illness, weather.	Risk score, likely reason, recommended inspection priority.
Heat detection	Activity, mounting/restlessness, cycle history, milk drop, previous breeding dates.	Probability of heat and recommended breeding window.
Weight prediction	Weight history, feed type, feed quantity, breed, age, health, season.	Expected market date, growth curve, underperformance warning.
Feed forecasting	Feed inventory, usage rate, herd size, ration plan, delivery lead time.	Days remaining, reorder date, cost forecast.
Water anomaly detection	Historical consumption, temperature, herd size, water level, flow rate.	Leak, pump failure, contamination risk, abnormal drinking alert.
Theft risk scoring	GPS movement, time of day, gate sensors, geofence, camera/RFID events.	Suspicious movement score and urgent incident alert.
Pasture condition support	Grazing duration, stocking rate, rainfall, drone/satellite/manual condition scores.	Move recommendation and recovery estimate.
Camera-based welfare detection	Video/images from gates, pens, water points, or barns.	Lameness, isolation, coughing pattern, body condition estimate, missing animal count.

Source notes: AI and PLF references [1], [2], [9].

10.2 AI safety and explainability rules

- Every AI alert should explain why it was created, for example: high temperature plus reduced movement plus reduced water intake.
- AI output should be a decision-support recommendation, not a final medical diagnosis.
- Farmers and vets should be able to confirm, reject, or comment on AI alerts.
- Confirmed and rejected alerts should become labeled feedback for future model improvement.
- The system should show confidence levels and avoid hiding uncertainty.
- High-risk medical or welfare alerts should encourage human inspection and vet involvement.

10.3 Feature engineering examples

Feature	Meaning	Example use
movement_last_6h_vs_baseline	Compares recent activity against animal normal behavior.	Reduced activity may indicate illness or injury.
water_intake_group_zscore	Compares water intake to group expectation.	Detect pump failure, heat stress, or disease.
weight_gain_14d	Recent average daily gain.	Identify underperforming animals.

Feature	Meaning	Example use
days_since_last_service	Time since mating or AI.	Pregnancy check scheduling and fertility analysis.
distance_from_allowed_zone	Geofence deviation distance.	Theft, escape, or device anomaly detection.
feed_cost_per_kg_gain	Feed spend divided by weight gain.	Profitability and ration adjustment.
temperature_humidity_index	Heat stress indicator for animals.	Heat stress alerts and cooling recommendations.

11. Alert Engine and Task Workflow

The alert engine is the part of the system that converts data into action. It should combine rule-based thresholds, schedules, AI scores, and escalation logic.

11.1 Alert severity levels

Severity	Meaning	Examples	Notification behavior
Low	Information or reminder.	Vaccine due in 7 days; feed report ready.	In-app notification, dashboard badge.
Medium	Needs attention but not immediately dangerous.	Water below 25%; animal weight gain below target.	Push notification and task creation.
High	Important operational or animal welfare risk.	Possible illness; vaccine overdue; pump failure.	Push plus SMS/WhatsApp; assign task.
Critical	Immediate risk of loss, death, theft, or major failure.	Animal exits geofence at night; water empty; severe heat stress.	Push, SMS/WhatsApp, email/call escalation, siren/gateway action where available.

11.2 Alert lifecycle

52. Detected: rule, schedule, or AI model identifies a condition.
53. Created: alert record is stored with severity, message, affected animal/location, evidence, and recommended action.
54. Notified: the right users receive alerts by the appropriate channels.
55. Assigned: a task is created for a worker, manager, or vet.
56. Acknowledged: a user confirms that they have seen it.
57. Investigated: field observation, photo, treatment, refill, repair, or movement action is recorded.
58. Resolved: alert is closed with outcome and resolution notes.
59. Learned: the system stores whether the alert was true, false, or uncertain.

11.3 Example alert rules

Rule	Condition	Action
Low water	water_level_percent < 20	Create medium alert and refill task.
Critical water	water_level_percent < 10 OR flow_rate = 0 for configured period	Create critical alert, notify owner/manager/worker.
Geofence breach	animal location outside allowed polygon for more than grace period	Create high or critical security alert.
Night movement	animal leaves zone between configured night	Create critical theft-risk alert.

Rule	Condition	Action
	hours	
Health risk	AI health score above threshold OR multiple abnormal indicators	Create health inspection task.
Vaccination due	next_due_date within reminder window	Create reminder and add to calendar.
Withdrawal warning	sale/milk record attempted before withdrawal_end_date	Block or warn depending on policy.
Device offline	no heartbeat within expected interval	Create maintenance alert.
Low battery	battery_level below threshold	Create device maintenance task.

12. Mobile App, Web Dashboard, and User Experience

The system should be easy for farmers and workers to use in the field, while still giving managers detailed analytics in the dashboard.

12.1 Mobile app screens

Screen	Purpose	Key features
Home dashboard	Today's operating summary.	Critical alerts, sick animals, due vaccines, tasks, water/feed status, animal count.
Animal list	Find and filter animals quickly.	Search by tag, RFID scan, species, group, health status, breeding status.
Animal profile	One animal's full digital record.	Timeline, photos, health, breeding, weight, production, location, notes.
Scan screen	Field identification.	RFID/QR scan, quick record entry, offline storage.
Tasks	Worker action list.	Assigned tasks, priority, due dates, photos, completion notes.
Alerts	Risk and reminder center.	Severity, affected animal/location, evidence, recommended action.
Map	Location and paddock overview.	Animal positions, geofences, paddocks, water points, gates.
Add record	Fast data capture.	Treatment, weight, feeding, water, breeding, birth, death, sale.
Reports	Simple summaries for mobile users.	Vaccination due, sick animals, weight gain, daily tasks.
Settings/offline	Connectivity and sync control.	Sync status, conflicts, cached farms, device pairing.

12.2 Web dashboard sections

- Farm overview: animal count, active alerts, herd status, device status, financial summary.
- Animal analytics: health, breeding, growth, production, mortality, sales, and traceability.
- Operations: feed, water, paddocks, tasks, workers, equipment, and reports.
- Map view: current locations, geofences, paddocks, water points, and security incidents.
- AI insights: top risks, recommended actions, underperforming animals, predicted feed shortages, and upcoming breeding events.
- Administration: users, roles, farms, devices, thresholds, integrations, subscriptions, and audit logs.

13. Security, Privacy, Auditability, and Data Protection

Security must be built into the platform from the beginning, especially because animal ownership, sales, financial information, and farm boundaries are sensitive.

Area	Requirement
Authentication	Use secure password hashing, optional MFA, session expiry, refresh tokens, and device trust controls.
Authorization	Role-based and farm-scoped access control for every API and database query.
Device security	Unique credentials per device, secure provisioning, key rotation, and revocation for lost devices.
Transport security	Use TLS for APIs and MQTT connections where possible.
Data encryption	Encrypt sensitive data at rest and use secure object-storage permissions.
Audit logs	Record sensitive actions, exports, ownership changes, deletions, medicine records, user changes, and support access.
Backups	Automated encrypted backups with restore testing.
Data retention	Configurable retention for telemetry, audit logs, media, and business records.
Privacy	Limit buyer/inspector access to selected traceability records, not full farm data.
Resilience	Rate limits, input validation, idempotency keys, retry logic, and monitoring.

14. Deployment, DevOps, Observability, and Maintenance

For a scalable product, the platform should use production-ready deployment practices, observability, and automated maintenance.

Component	Recommended approach
Application hosting	Containerized backend services running on Kubernetes for larger deployments, or managed containers/serverless for early-stage deployment.
Database	Managed PostgreSQL with PostGIS, backups, read replicas where needed, and partitioning for large tables.
Telemetry storage	TimescaleDB or another time-series storage strategy with retention policies and continuous aggregates.
Messaging	MQTT broker for IoT data; Kafka/Redpanda/NATS optional for larger event streaming.
File storage	S3-compatible object storage for photos, videos, reports, certificates, and documents.
Observability	OpenTelemetry traces/metrics/logs, dashboards, uptime checks, error tracking, and alerting.
CI/CD	Automated tests, linting, Docker builds, database migrations, deployment pipelines, and rollback strategy.
Infrastructure as code	Terraform/OpenTofu or cloud-native templates for repeatable deployment.
Monitoring metrics	API latency, ingestion lag, MQTT message rate, device offline rate, failed syncs, alert delivery success, database load.
Maintenance	Firmware update planning, device replacement workflow, calibration

Component	Recommended approach
	schedules, data-quality audits, and backup restore tests.

Source notes: *Kubernetes* [10], *OpenTelemetry* [11], *Kafka* [8].

15. Implementation Roadmap

The system should be implemented in phases so that it becomes useful quickly and can grow without major redesign.

Phase	Build scope	Outcome
Phase 1: Core digital records	Users, farms, animals, RFID/QR, health, vaccination, breeding, weight, tasks, basic reports, offline mobile capture.	Useful farm management system without expensive hardware.
Phase 2: IoT starter kit	Device registry, gateway, MQTT ingestion, water sensors, feed sensors, digital scale, GPS for selected animals, basic alerts.	Automatic monitoring for key farm risks.
Phase 3: AI and recommendations	Health risk scoring, heat detection, weight prediction, feed forecasting, theft-risk scoring, improved dashboard.	Decision-support platform that recommends action.
Phase 4: Advanced operations	Drone/camera integration, pasture analytics, production analytics, traceability portal, abattoir/buyer export, insurance reports.	Commercial-grade livestock intelligence platform.
Phase 5: Scale and ecosystem	Multi-farm organizations, cooperative dashboards, API integrations, regional disease alerts, marketplace integrations.	Platform can support many farms and external partners.

15.1 Minimum viable product

- Animal registration with RFID/QR support.
- Health, vaccination, treatment, breeding, weight, birth, sale, and death records.
- Mobile app with offline data capture and sync.
- Basic web dashboard and reports.
- Alert/reminder engine for vaccinations, follow-ups, breeding dates, and low stock/manual water checks.
- User roles: owner, manager, worker, vet.
- Simple database model that is ready for IoT expansion.

15.2 Recommended first hardware integration

- RFID reader and RFID ear tags for reliable animal identification.
- Digital weighing scale with animal ID linking.
- Water-level sensor for critical water points.
- Feed-bin load cell or level sensor for stock monitoring.
- GSM/LoRaWAN gateway with solar backup.
- GPS collars for high-value animals before expanding to the full herd.

16. Claude-Ready System Prompt

The following prompt can be pasted into Claude or another AI coding/design assistant to guide detailed system design or implementation planning.

Claude system prompt

You are a senior software architect, IoT systems engineer, database designer, AI product strategist, and livestock opera
Design a complete modern livestock management platform for farms in rural and commercial environments.

The platform must support animal identification, RFID/QR registration, GPS and smart collar integration, health records,

Design the solution with these principles:

1. Offline-first field operation for farms with poor connectivity.
2. Modular backend that can start as a modular monolith but scale to microservices.
3. Event-driven logic for telemetry, alerts, task creation, and reporting.
4. PostgreSQL for core records, PostGIS for geospatial data, TimescaleDB or equivalent for telemetry, Redis for cache, o
5. Security through farm-scoped RBAC, device credentials, TLS, audit logs, backups, and encrypted storage.
6. AI as decision support, not final veterinary diagnosis.
7. Clear user experience for farm owners, managers, workers, vets, admins, buyers, and inspectors.

Produce:

- Full architecture.
 - Database schema.
 - API design.
 - MQTT topic design.
 - Alert rules.
 - AI model logic.
 - Offline sync logic.
 - User roles and permissions.
 - Mobile app screens.
 - Web dashboard structure.
 - Deployment plan.
 - Implementation phases.
 - Example payloads and pseudocode.
- Make the design practical, scalable, and suitable for African livestock farming conditions.

17. Appendices

17.1 Example API endpoints

Method	Endpoint	Purpose
POST	/api/v1/farms	Create farm.
GET	/api/v1/farms/{farmId}/dashboard	Get farm summary dashboard.
POST	/api/v1/animals	Register animal.
GET	/api/v1/animals/{animalId}	Get animal profile and timeline.
POST	/api/v1/animals/{animalId}/health-records	Create treatment/health record.
POST	/api/v1/animals/{animalId}/weight-records	Create weight record.
POST	/api/v1/breeding-records	Create mating/AI/pregnancy record.
POST	/api/v1/devices	Register device.
POST	/api/v1/telemetry/batch	Gateway/mobile batch telemetry upload.
GET	/api/v1/alerts	List alerts by farm, severity, status, type.

Method	Endpoint	Purpose
PATCH	/api/v1/alerts/{alertId}/acknowledge	Acknowledge alert.
PATCH	/api/v1/tasks/{taskId}/complete	Complete task with notes and evidence.
GET	/api/v1/reports/health	Generate or retrieve health report.
POST	/api/v1/sync/push	Push offline mobile changes.
GET	/api/v1/sync/pull	Pull server changes since last sync token.

17.2 Example telemetry payload

```
{
  "message_id": "01HZ-FARM-123-000045",
  "farm_id": "farm_123",
  "gateway_id": "gw_north_001",
  "device_id": "water_sensor_014",
  "device_type": "water_level_sensor",
  "metric": "water_level_percent",
  "value": 8.4,
  "unit": "%",
  "location_id": "water_point_b",
  "timestamp": "2026-05-15T08:40:00Z",
  "battery_percent": 73,
  "signal_rssi": -96,
  "raw": {"adc": 312, "firmware": "1.2.4"}
}
```

17.3 Example alert object

```
{
  "alert_id": "alert_789",
  "farm_id": "farm_123",
  "type": "water_low",
  "severity": "critical",
  "title": "Critical water shortage at Paddock B",
  "message": "Water Point B is at 8.4%. Refill immediately or inspect pump/pipe failure.",
  "affected_entity_type": "water_point",
  "affected_entity_id": "water_point_b",
  "evidence": {
    "last_reading": 8.4,
    "threshold": 10,
    "device_id": "water_sensor_014"
  },
  "recommended_action": "Assign worker to refill tank and inspect pump.",
  "status": "open",
  "created_at": "2026-05-15T08:40:05Z"
}
```

17.4 Example pseudocode for telemetry processing

```
function processTelemetry(payload):
  authenticateDevice(payload.device_id, payload.gateway_id)
  validateSchema(payload)
  farm = findFarm(payload.farm_id)
  device = findDevice(payload.device_id, farm.id)
  normalized = normalizeUnits(payload)
  saveRawPayload(payload)
```

```

saveTelemetryReading(normalized)
updateDeviceLastSeen(device, payload.timestamp, payload.battery_percent)

event = publishEvent("device.telemetry.received", normalized)
ruleResults = rulesEngine.evaluate(event)
aiResults = aiEngine.scoreIfRelevant(event)

for result in ruleResults + aiResults:
    if result.requires_alert:
        alert = createAlert(result)
        createTaskIfNeeded(alert)
        notifyUsers(alert)

updateDashboardCache(farm.id)
return success

```

17.5 Example database DDL sketch

```

CREATE TABLE farms (
  farm_id UUID PRIMARY KEY,
  organization_id UUID NOT NULL,
  name TEXT NOT NULL,
  timezone TEXT NOT NULL DEFAULT 'Africa/Harare',
  boundary GEOGRAPHY(POLYGON, 4326),
  status TEXT NOT NULL DEFAULT 'active',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE TABLE animals (
  animal_id UUID PRIMARY KEY,
  farm_id UUID NOT NULL REFERENCES farms(farm_id),
  tag_number TEXT NOT NULL,
  rfid_code TEXT,
  species TEXT NOT NULL,
  breed TEXT,
  sex TEXT CHECK (sex IN ('male', 'female', 'unknown')),
  date_of_birth DATE,
  status TEXT NOT NULL DEFAULT 'active',
  health_status TEXT NOT NULL DEFAULT 'normal',
  reproductive_status TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (farm_id, tag_number),
  UNIQUE (farm_id, rfid_code)
);

```

```

CREATE TABLE alerts (
  alert_id UUID PRIMARY KEY,
  farm_id UUID NOT NULL REFERENCES farms(farm_id),
  animal_id UUID REFERENCES animals(animal_id),
  alert_type TEXT NOT NULL,
  severity TEXT NOT NULL CHECK (severity IN ('low', 'medium', 'high', 'critical')),
  title TEXT NOT NULL,
  message TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'open',
  evidence JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  resolved_at TIMESTAMPTZ
);

```

);

17.6 Key performance indicators

Category	KPI
Animal health	Disease incidence, mortality rate, recovery time, treatment success rate.
Breeding	Conception rate, calving/kidding/lambing interval, open days, birth survival rate.
Growth	Average daily gain, market readiness, underperforming animal count.
Feed	Feed conversion ratio, feed cost per kg gain, days of feed remaining.
Water	Water availability uptime, abnormal consumption alerts, leak incidents.
Security	Geofence breaches, theft incidents, response time, missing animal recovery rate.
Finance	Profit per animal, cost per animal, revenue per production unit, total farm margin.
Operations	Task completion rate, alert response time, device uptime, data sync success rate.

17.7 References

- [1] Precision livestock management in the Internet of Things era - ScienceDirect.
<https://www.sciencedirect.com/science/article/pii/S0190052823000524>
- [2] Precision Livestock Farming Technology: Applications and Challenges - MDPI Agriculture. <https://www.mdpi.com/2077-0472/14/4/620>
- [3] LoRa Alliance technical specifications. <https://resources.lora-alliance.org/technical-specifications>
- [4] Semtech LoRaWAN Standard overview. <https://www.semtech.com/lora/lorawan-standard>
- [5] MQTT Version 5.0 OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [6] PostGIS official site. <https://postgis.net/>
- [7] TimescaleDB official site. <https://timescaledb.org/>
- [8] Apache Kafka official site. <https://kafka.apache.org/>
- [9] Artificial Intelligence and Sensor Innovations: Enhancing Livestock Welfare - Springer.
<https://link.springer.com/article/10.1007/s44230-023-00050-2>
- [10] Kubernetes official site. <https://kubernetes.io/>
- [11] OpenTelemetry CNCF project page. <https://www.cncf.io/projects/opentelemetry/>

End of document